

MamaCare — Product Requirements Document

Maternal Health Appointment Tracker

1. Executive Summary

MamaCare is an offline-first Progressive Web Application (PWA) designed for community health workers (CHWs) in Sierra Leone to register expectant mothers, log visits, schedule appointments, flag at-risk patients, and access a rule-based health decision tree for common symptoms. The application syncs to a cloud backend when connectivity is available, and includes a district nurse dashboard for remote case review.

Key Stats Driving This Product:

- Sierra Leone has one of the world’s highest maternal mortality rates
- 38% of health facilities have no electricity
- Community health workers in rural areas lack digital tools and doctor consultation access

2. Problem Statement

2.1 Current Pain Points

Stakeholder	Pain Point
Community Health Workers (CHWs)	No doctor to consult in rural areas; carry heavy paper protocols
Pregnant Women	Miss clinic appointments due to distance, forgetfulness, or lack of reminders
District Nurses	No visibility into CHW activities or patient status remotely
Health System	No centralized data for tracking maternal health outcomes

2.2 Impact

- Delayed or missed prenatal care
- Unidentified high-risk pregnancies
- Higher maternal and infant mortality
- Inefficient use of limited healthcare resources

3. Product Vision & Goals

3.1 Vision

Empower community health workers with a reliable, offline-capable digital tool to ensure no pregnant woman is left without care, regardless of infrastructure limitations.

3.2 Goals

Goal	Metric
Reduce missed prenatal appointments	<10% missed appointments
Improve early risk identification	100% of high-risk patients flagged within 24 hours
Enable remote oversight	District nurses can review 100% of cases within 48 hours
Ensure data availability	99.9% uptime for offline functionality

4. Target Users & Personas

4.1 Primary User: Community Health Worker (CHW)

- **Demographics:** Rural Sierra Leone, limited formal education, basic smartphone literacy
- **Environment:** No reliable electricity, intermittent mobile data
- **Needs:** Simple interface, works offline, quick data entry, decision support
- **Device:** Low-end Android smartphone

4.2 Secondary User: District Nurse

- **Demographics:** Supervisory role, based at district health center
- **Environment:** Intermittent connectivity, laptop/desktop access
- **Needs:** Dashboard view of all patients, at-risk alerts, ability to review CHW work
- **Device:** Laptop or tablet with better connectivity

4.3 Tertiary User: Pregnant Woman

- **Demographics:** Rural, may be illiterate, limited phone access
- **Needs:** Appointment reminders via SMS/WhatsApp, clear communication
- **Device:** Basic feature phone or shared family phone

5. Functional Requirements

5.1 Authentication (Week 1)

FR-AUTH-001: CHWs must authenticate via phone number + OTP or username/password. **FR-AUTH-002:** District nurses authenticate with elevated role permissions. **FR-AUTH-003:** Session persists locally (offline) and validates on sync. **FR-AUTH-004:** Support for single-device-per-user to prevent data conflicts.

5.2 Patient Registration (Week 1)

FR-REG-001: Register expectant mothers with:

- Full name
- Age
- Estimated delivery date (EDD)
- Phone number (for reminders)

- Address / village
- Emergency contact
- Pregnancy history (gravida/para)
- Known risk factors (pre-eclampsia, diabetes, previous C-section, etc.)

FR-REG-002: Generate unique patient ID automatically. **FR-REG-003:** Capture patient photo (optional, for identification). **FR-REG-004:** All registration data stored in IndexedDB immediately.

5.3 Visit Logging (Week 1)

FR-VIS-001: Log prenatal visits with:

- Date and time (auto-captured)
- Blood pressure
- Weight
- Fundal height
- Fetal heart rate
- Urine protein/glucose
- Symptoms reported
- Medications given (iron, folate, malaria prophylaxis)
- CHW notes

FR-VIS-002: Support for home visits and clinic visits. **FR-VIS-003:** View patient visit history chronologically. **FR-VIS-004:** Offline-first: all visits stored locally, sync on connectivity.

5.4 Appointment Scheduling (Week 1)

FR-APT-001: Schedule next appointment with:

- Date and time
- Location (home visit or clinic)
- Purpose (routine check, follow-up, referral)
- Reminder settings

FR-APT-002: View upcoming appointments in calendar/list view. **FR-APT-003:** Mark appointments as completed, missed, or rescheduled. **FR-APT-004:** Auto-calculate recommended visit intervals based on gestational age.

5.5 Decision Tree for Symptoms (Week 1)

FR-DT-001: Rule-based decision tree for common maternal symptoms:

- Headache + vision changes → Pre-eclampsia warning
- Severe abdominal pain → Emergency referral
- Fever → Malaria/sepsis screening

- Reduced fetal movement → Urgent assessment
- Vaginal bleeding → Emergency protocol
- Swelling + high BP → Pre-eclampsia protocol

FR-DT-002: Each decision tree provides:

- Triage level (green/yellow/red)
- Recommended action
- Referral necessity
- Documentation prompts

FR-DT-003: Decision tree logic embedded in app (works offline). **FR-DT-004:** Results logged with visit record.

5.6 At-Risk Patient Flagging (Week 2)

FR-RISK-001: Auto-flag patients as at-risk when:

- Missed 2+ consecutive appointments
- Decision tree indicates red/yellow triage
- Vital signs exceed thresholds (BP >140/90, etc.)
- Known risk factors present

FR-RISK-002: Risk levels: Low (green), Medium (yellow), High (red). **FR-RISK-003:** At-risk patients appear in prioritized dashboard view. **FR-RISK-004:** CHW receives alert when patient becomes at-risk. **FR-RISK-005:** Risk status syncs to district nurse dashboard.

5.7 District Nurse Dashboard (Week 2)

FR-DASH-001: Overview of all patients in district with:

- Total registered patients
- Upcoming appointments (next 7 days)
- Missed appointments
- At-risk patient count by severity

FR-DASH-002: Patient detail view with full history. **FR-DASH-003:** Filter by CHW, village, risk level, appointment status. **FR-DASH-004:** Export reports (CSV/PDF) for district health meetings. **FR-DASH-005:** Ability to send messages/notes to CHWs.

5.8 Firebase Sync (Week 2)

FR-SYNC-001: Automatic background sync when connectivity detected. **FR-SYNC-002:** Conflict resolution: server wins for nurse edits, device wins for CHW edits (last-write-wins with timestamp). **FR-SYNC-003:** Sync queue with retry logic for failed uploads. **FR-SYNC-004:** Offline indicator showing sync status. **FR-SYNC-005:** Support for partial sync (sync what you can, queue the rest).

5.9 SMS/WhatsApp Reminders (Week 2)

FR-REM-001: Automated reminders sent 24 hours and 2 hours before appointments. **FR-REM-002:** Reminder content in simple English (Krio Phase 2). **FR-REM-003:** Reminder includes: date, time, location, what to bring. **FR-REM-004:** Delivery status tracking (sent/delivered/failed). **FR-REM-005:** Fallback: if WhatsApp fails, try SMS.

6. Non-Functional Requirements

6.1 Performance

Requirement	Target
App load time (first visit)	<3 seconds on 3G
Page transitions	<500ms
Offline data operations	<200ms
Sync time (per record)	<2 seconds
Supported concurrent users	500+ CHWs per district

6.2 Reliability

- App must function fully without internet for 30+ days
- Data loss rate: 0% (all operations queued until sync)
- Battery efficient: background sync only on charging or user-initiated

6.3 Accessibility

- Large touch targets (min 48x48dp)
- High contrast mode
- Simple language (Grade 6 reading level)
- Voice input support where possible
- Works on screen sizes 320px+

6.4 Security & Privacy

- All patient data encrypted at rest (IndexedDB encryption)
- HTTPS for all network communications
- Role-based access control (CHW vs Nurse)
- PHI compliance (health data protection)
- No patient data stored in plain text
- Auto-logout after 15 minutes of inactivity

6.5 Compatibility

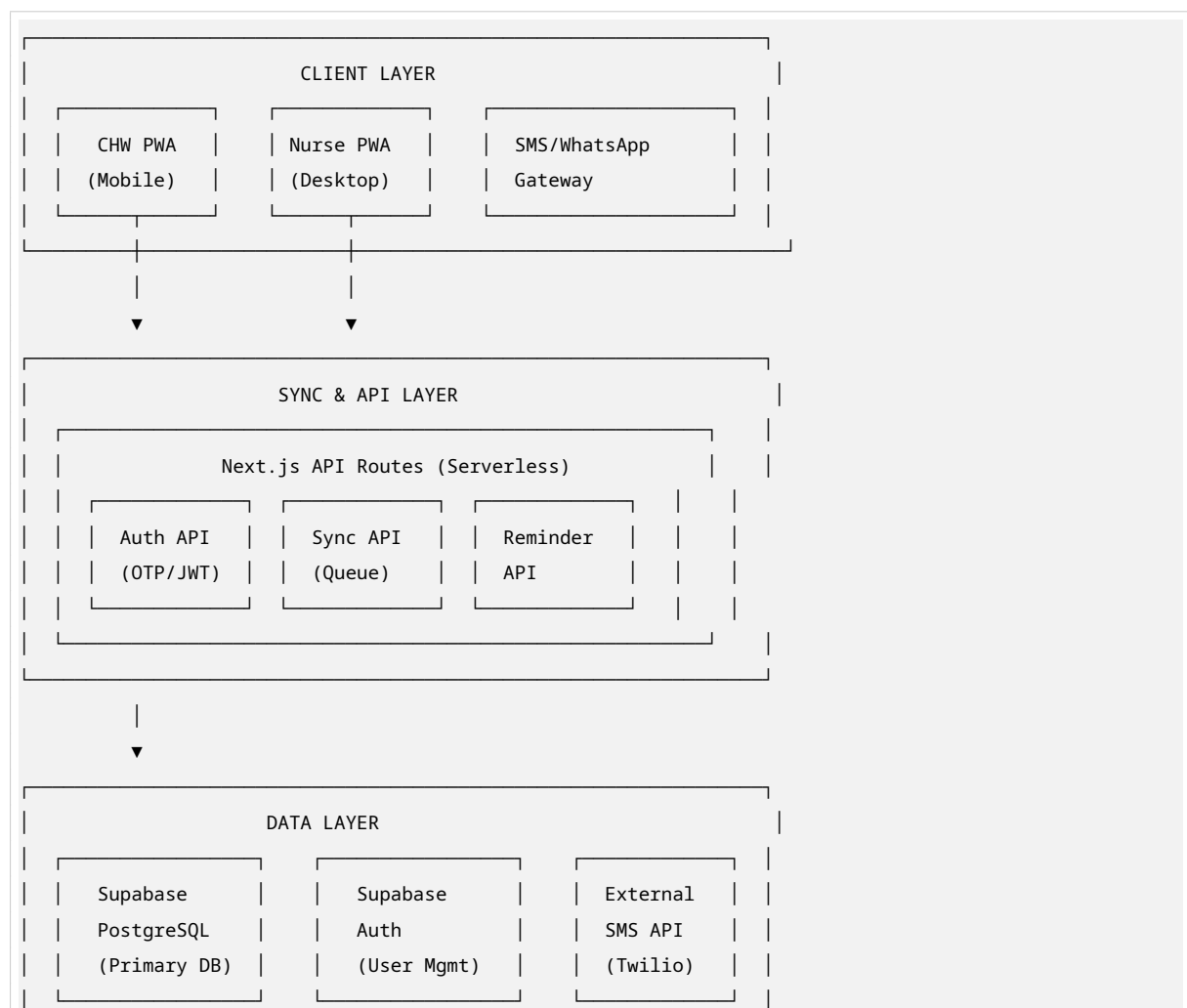
- Android 8.0+ (API 26+)
 - iOS 14+ (if applicable)
 - Chrome, Firefox, Safari mobile browsers
 - Works on devices with 2GB RAM
-

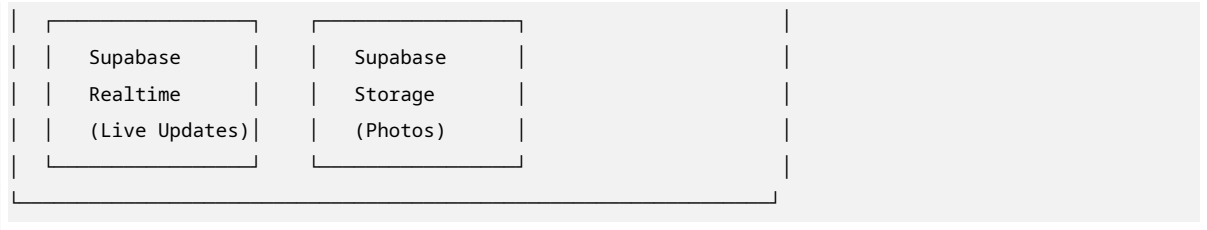
7. Out of Scope (Phase 1)

Feature	Reason	Planned Phase
Full EMR integration	Requires government partnership	Phase 3
Government approval/regulatory	Lengthy process, not blocking MVP	Phase 2
Multi-language Krio support	Translation pipeline needed	Phase 2
Video telemedicine	Bandwidth requirements too high	Phase 3
Integration with national health ID	Requires government API	Phase 3
Advanced analytics/ML risk prediction	Needs data accumulation first	Phase 3

8. Technical Design

8.1 System Architecture



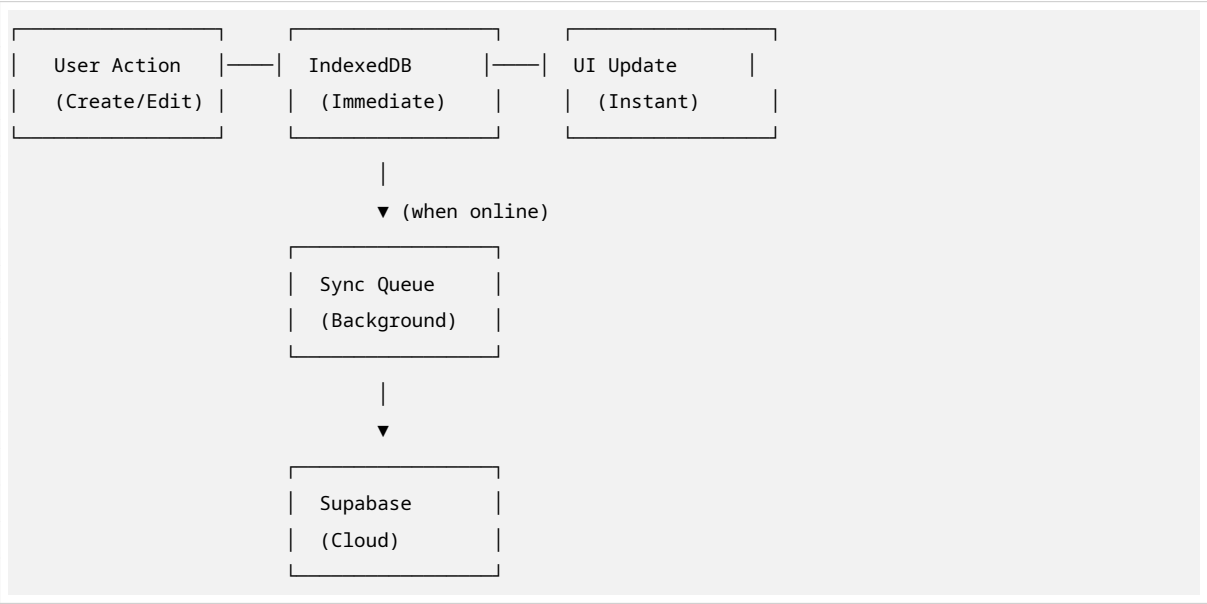


8.2 Tech Stack Justification

Technology	Purpose	Justification
Next.js 14+	Frontend framework	SSR/SSG for fast loads, API routes for backend, excellent PWA support
Supabase	Backend-as-a-Service	PostgreSQL + Auth + Realtime + Storage in one platform, generous free tier, open source
IndexedDB (via idb-keyval or Dexie.js)	Local offline storage	Native browser API, large storage capacity, works without internet
date-fns	Date manipulation	Lightweight, tree-shakeable, excellent for scheduling logic
Vercel	Hosting	Zero-config deployments, edge network, excellent Next.js integration
Service Workers	PWA offline support	Cache assets, intercept network requests, background sync
Workbox	Service worker management	Google's library for reliable caching strategies

8.3 Data Architecture

8.3.1 Offline-First Strategy



8.3.2 IndexedDB Schema (Local)

```

// Database: MamaCareDB
// Version: 1

const stores = {
  patients: {
    keyPath: 'id',
    indexes: ['phone', 'chw_id', 'risk_level', 'edd']
  },
  visits: {
    keyPath: 'id',
    indexes: ['patient_id', 'visit_date', 'chw_id']
  },
  appointments: {
    keyPath: 'id',
    indexes: ['patient_id', 'appointment_date', 'status', 'chw_id']
  },
  syncQueue: {
    keyPath: 'id',
    indexes: ['table', 'operation', 'status', 'created_at']
  },
  decisionTreeLogs: {
    keyPath: 'id',
    indexes: ['patient_id', 'visit_id', 'triage_level']
  },
  users: {
    keyPath: 'id',
    indexes: ['phone', 'role']
  }
};

```

8.3.3 Supabase Schema (Cloud) Table: patients

```

CREATE TABLE patients (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chw_id UUID REFERENCES auth.users(id),
  full_name TEXT NOT NULL,
  age INTEGER,
  phone TEXT,
  village TEXT,
  address TEXT,
  emergency_contact TEXT,
  emergency_phone TEXT,
  edd DATE, -- Estimated Delivery Date
  gravida INTEGER DEFAULT 0,
  para INTEGER DEFAULT 0,
  known_risk_factors JSONB DEFAULT '[]',
  risk_level TEXT DEFAULT 'low' CHECK (risk_level IN ('low', 'medium', 'high')),
  photo_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

```

```

    synced_at TIMESTAMPTZ,
    is_deleted BOOLEAN DEFAULT FALSE
);

CREATE INDEX idx_patients_chw ON patients(chw_id);
CREATE INDEX idx_patients_risk ON patients(risk_level);
CREATE INDEX idx_patients_edd ON patients(edd);

```

Table: visits

```

CREATE TABLE visits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID REFERENCES patients(id),
  chw_id UUID REFERENCES auth.users(id),
  visit_date TIMESTAMPTZ DEFAULT NOW(),
  visit_type TEXT CHECK (visit_type IN ('home', 'clinic')),
  blood_pressure_systolic INTEGER,
  blood_pressure_diastolic INTEGER,
  weight_kg DECIMAL(5,2),
  fundal_height_cm DECIMAL(5,2),
  fetal_heart_rate INTEGER,
  urine_protein TEXT,
  urine_glucose TEXT,
  symptoms TEXT[],
  medications_given JSONB DEFAULT '[]',
  chw_notes TEXT,
  decision_tree_result JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  synced_at TIMESTAMPTZ,
  is_deleted BOOLEAN DEFAULT FALSE
);

CREATE INDEX idx_visits_patient ON visits(patient_id);
CREATE INDEX idx_visits_date ON visits(visit_date);

```

Table: appointments

```

CREATE TABLE appointments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  patient_id UUID REFERENCES patients(id),
  chw_id UUID REFERENCES auth.users(id),
  appointment_date TIMESTAMPTZ NOT NULL,
  location TEXT,
  purpose TEXT,
  status TEXT DEFAULT 'scheduled' CHECK (status IN ('scheduled', 'completed', 'missed', 'rescheduled',
    ⇐ 'cancelled')),
  reminder_sent_24h BOOLEAN DEFAULT FALSE,
  reminder_sent_2h BOOLEAN DEFAULT FALSE,
  reminder_status TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

```

```

    synced_at TIMESTAMPTZ,
    is_deleted BOOLEAN DEFAULT FALSE
);

CREATE INDEX idx_appointments_patient ON appointments(patient_id);
CREATE INDEX idx_appointments_date ON appointments(appointment_date);
CREATE INDEX idx_appointments_status ON appointments(status);

```

Table: decision_tree_rules

```

CREATE TABLE decision_tree_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rule_name TEXT NOT NULL,
    symptoms JSONB NOT NULL, -- Array of symptom conditions
    triage_level TEXT CHECK (triage_level IN ('green', 'yellow', 'red')),
    action_required TEXT,
    referral_needed BOOLEAN DEFAULT FALSE,
    priority INTEGER DEFAULT 0,
    version INTEGER DEFAULT 1,
    is_active BOOLEAN DEFAULT TRUE
);

```

Table: sync_logs

```

CREATE TABLE sync_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    device_id TEXT,
    chw_id UUID REFERENCES auth.users(id),
    sync_started_at TIMESTAMPTZ,
    sync_completed_at TIMESTAMPTZ,
    records_pushed INTEGER DEFAULT 0,
    records_pulled INTEGER DEFAULT 0,
    conflicts INTEGER DEFAULT 0,
    errors JSONB DEFAULT '[]',
    status TEXT CHECK (status IN ('success', 'partial', 'failed'))
);

```

Row Level Security (RLS) Policies:

```

-- CHWs can only see their own patients
CREATE POLICY "CHW sees own patients" ON patients
    FOR ALL USING (auth.uid() = chw_id);

-- Nurses can see all patients in their district
CREATE POLICY "Nurse sees district patients" ON patients
    FOR SELECT USING (
        EXISTS (
            SELECT 1 FROM user_roles
            WHERE user_id = auth.uid()
            AND role = 'nurse'
        )
    );

```

```
-- Similar policies for visits and appointments
```

8.4 Application Architecture

8.4.1 Frontend Structure

```
app/
├─ (auth)/
│  ├─ login/
│  │   └─ page.tsx          # Phone/OTP login
│  └─ layout.tsx
├─ (dashboard)/
│  ├─ chw/
│  │   ├─ page.tsx          # CHW home dashboard
│  │   └─ patients/
│  │       ├─ page.tsx      # Patient list
│  │       └─ new/
│  │           └─ page.tsx  # Register new patient
│  │               └─ [id]/
│  │                   └─ page.tsx # Patient detail
│  │   └─ visits/
│  │       └─ new/
│  │           └─ page.tsx  # Log new visit
│  │               └─ [id]/
│  │                   └─ page.tsx # Visit detail
│  │   └─ appointments/
│  │       ├─ page.tsx      # Appointment calendar
│  │       └─ new/
│  │           └─ page.tsx  # Schedule appointment
│  │   └─ decision-tree/
│  │       └─ page.tsx      # Symptom checker
│  │   └─ at-risk/
│  │       └─ page.tsx      # At-risk patients view
│  └─ nurse/
│      ├─ page.tsx          # Nurse dashboard
│      └─ patients/
│          └─ page.tsx      # All district patients
│      └─ reports/
│          └─ page.tsx      # Export reports
├─ api/
│  ├─ auth/
│  │   └─ otp/
│  │       └─ route.ts      # OTP generation/verification
│  └─ sync/
│      └─ route.ts          # Bidirectional sync endpoint
│  └─ reminders/
│      └─ route.ts          # Trigger reminder sending
├─ components/
│  └─ ui/                  # Reusable UI components
│  └─ forms/               # Form components
```

```

|   ├── charts/           # Data visualization
|   └── offline/         # Offline indicators
├── hooks/
|   ├── useOfflineStorage.ts # IndexedDB operations
|   ├── useSync.ts         # Sync logic
|   ├── useAuth.ts         # Authentication
|   └── useDecisionTree.ts  # Decision tree engine
├── lib/
|   ├── db/
|   |   ├── indexeddb.ts   # IndexedDB setup
|   |   └── supabase.ts    # Supabase client
|   ├── sync/
|   |   ├── engine.ts      # Sync engine
|   |   └── conflict.ts    # Conflict resolution
|   ├── decision-tree/
|   |   ├── rules.ts       # Decision tree rules
|   |   └── engine.ts      # Rule evaluation engine
|   └── utils/
|       └── date.ts        # Date utilities (date-fns)
├── types/
|   └── index.ts           # TypeScript interfaces
├── public/
|   ├── manifest.json      # PWA manifest
|   └── sw.js              # Service worker (Workbox)
└── middleware.ts          # Route protection

```

8.4.2 Key Components OfflineStorage Service:

```

// lib/db/indexeddb.ts

import { openDB, DBSchema } from 'idb';

interface MamaCareDB extends DBSchema {
  patients: {
    key: string;
    value: Patient;
    indexes: { 'by-chw': string; 'by-risk': string };
  };
  syncQueue: {
    key: string;
    value: SyncOperation;
    indexes: { 'by-status': string };
  };
}

export const db = openDB<MamaCareDB>('MamaCareDB', 1, {
  upgrade(db) {
    const patientStore = db.createObjectStore('patients', { keyPath: 'id' });
    patientStore.createIndex('by-chw', 'chw_id');
    patientStore.createIndex('by-risk', 'risk_level');
  }
});

```

```

    const syncStore = db.createObjectStore('syncQueue', { keyPath: 'id' });
    syncStore.createIndex('by-status', 'status');
  }
});

export async function addToSyncQueue(operation: SyncOperation) {
  const database = await db;
  await database.add('syncQueue', {
    ...operation,
    status: 'pending',
    created_at: new Date().toISOString(),
    retry_count: 0
  });
}

```

Sync Engine:

```

// lib/sync/engine.ts

export class SyncEngine {
  private isOnline: boolean = navigator.onLine;
  private syncInProgress: boolean = false;

  constructor() {
    window.addEventListener('online', () => this.handleOnline());
    window.addEventListener('offline', () => this.handleOffline());
  }

  async sync() {
    if (this.syncInProgress || !this.isOnline) return;
    this.syncInProgress = true;

    try {
      // 1. Push local changes to server
      await this.pushChanges();

      // 2. Pull server changes
      await this.pullChanges();

      // 3. Resolve conflicts
      await this.resolveConflicts();

    } catch (error) {
      console.error('Sync failed:', error);
    } finally {
      this.syncInProgress = false;
    }
  }

  private async pushChanges() {
    const database = await db;
  }
}

```

```

const pending = await database.getAllFromIndex('syncQueue', 'by-status', 'pending');

for (const operation of pending) {
  try {
    await this.executeOperation(operation);
    await database.delete('syncQueue', operation.id);
  } catch (error) {
    await this.handleSyncError(operation, error);
  }
}

private async executeOperation(operation: SyncOperation) {
  switch (operation.table) {
    case 'patients':
      return supabase.from('patients').upsert(operation.data);
    case 'visits':
      return supabase.from('visits').upsert(operation.data);
    case 'appointments':
      return supabase.from('appointments').upsert(operation.data);
  }
}

private async handleOnline() {
  this.isOnline = true;
  await this.sync();
}

private handleOffline() {
  this.isOnline = false;
}
}

```

Decision Tree Engine:

```

// lib/decision-tree/engine.ts

interface SymptomInput {
  symptoms: string[];
  vitalSigns: {
    systolic?: number;
    diastolic?: number;
    temperature?: number;
  };
  gestationalAge?: number;
}

interface TriageResult {
  level: 'green' | 'yellow' | 'red';
  action: string;
  referralNeeded: boolean;
}

```

```

priority: number;
documentation: string[];
}

export function evaluateDecisionTree(input: SymptomInput): TriageResult {
  const { symptoms, vitalSigns } = input;

  // Red flags - Emergency
  if (symptoms.includes('vaginal_bleeding') ||
    symptoms.includes('severe_abdominal_pain') ||
    (vitalSigns.systolic && vitalSigns.systolic >= 160) ||
    (vitalSigns.diastolic && vitalSigns.diastolic >= 110)) {
    return {
      level: 'red',
      action: 'IMMEDIATE REFERRAL to nearest health facility. Do not delay.',
      referralNeeded: true,
      priority: 1,
      documentation: ['Record time of onset', 'Estimate blood loss', 'Check consciousness level']
    };
  }

  // Yellow flags - Urgent
  if ((symptoms.includes('headache') && symptoms.includes('vision_changes')) ||
    (symptoms.includes('swelling') && vitalSigns.systolic && vitalSigns.systolic >= 140) ||
    symptoms.includes('reduced_fetal_movement')) {
    return {
      level: 'yellow',
      action: 'Refer to clinic within 24 hours. Monitor closely.',
      referralNeeded: true,
      priority: 2,
      documentation: ['BP trend', 'Urine protein if available', 'Fetal movement count']
    };
  }

  // Green - Routine
  return {
    level: 'green',
    action: 'Continue routine care. Schedule next visit as planned.',
    referralNeeded: false,
    priority: 3,
    documentation: ['General wellbeing', 'Nutrition counseling given']
  };
}

```

8.5 API Design

8.5.1 Authentication

```

POST /api/auth/otp/request
Body: { phone: string }
Response: { success: boolean, message: string }

```

```
POST /api/auth/otp/verify
Body: { phone: string, otp: string }
Response: { token: string, user: User, role: string }
```

8.5.2 Sync

```
POST /api/sync/push
Headers: Authorization: Bearer <token>
Body: { operations: SyncOperation[] }
Response: {
  processed: number,
  failed: number,
  conflicts: Conflict[],
  serverTimestamp: string
}

GET /api/sync/pull
Headers: Authorization: Bearer <token>
Query: { lastSync: ISOString, tables: string[] }
Response: {
  patients: Patient[],
  visits: Visit[],
  appointments: Appointment[],
  deletedIds: string[],
  serverTimestamp: string
}
```

8.5.3 Reminders

```
POST /api/reminders/send
Headers: Authorization: Bearer <token>
Body: {
  appointmentId: string,
  type: '24h' | '2h',
  channel: 'sms' | 'whatsapp'
}
Response: { success: boolean, messageId?: string, error?: string }
```

8.6 PWA Configuration

manifest.json:

```
{
  "name": "MamaCare - Maternal Health Tracker",
  "short_name": "MamaCare",
  "description": "Offline-first maternal health appointment tracker for community health workers",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#e11d48",
```

```

"orientation": "portrait",
"icons": [
  {
    "src": "/icons/icon-72x72.png",
    "sizes": "72x72",
    "type": "image/png"
  },
  {
    "src": "/icons/icon-192x192.png",
    "sizes": "192x192",
    "type": "image/png"
  },
  {
    "src": "/icons/icon-512x512.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any maskable"
  }
]
}

```

Service Worker Strategy:

```

// public/sw.js (Workbox generated)

// Cache-first for static assets
workbox.routing.registerRoute(
  ({request}) => request.destination === 'script' ||
    request.destination === 'style' ||
    request.destination === 'image',
  new workbox.strategies.CacheFirst({
    cacheName: 'assets',
    plugins: [
      new workbox.expiration.ExpirationPlugin({
        maxEntries: 100,
        maxAgeSeconds: 30 * 24 * 60 * 60 // 30 days
      })
    ]
  })
);

// Network-first for API calls (with fallback)
workbox.routing.registerRoute(
  ({url}) => url.pathname.startsWith('/api/'),
  new workbox.strategies.NetworkFirst({
    cacheName: 'api-cache',
    plugins: [
      new workbox.backgroundSync.BackgroundSyncPlugin('sync-queue', {
        maxRetentionTime: 24 * 60 // 24 hours
      })
    ]
  })
);

```

```

    })
  );

  // Background sync for offline mutations
  workbox.backgroundSync.registerRoute(
    '/api/sync/push',
    new workbox.backgroundSync.Plugin('sync-queue', {
      maxRetentionTime: 24 * 60
    }),
    'POST'
  );

```

8.7 State Management

```

// Using React Context + useReducer for global state

interface AppState {
  isOnline: boolean;
  isSyncing: boolean;
  lastSyncTime: Date | null;
  pendingSyncCount: number;
  currentUser: User | null;
  patients: Patient[];
  atRiskPatients: Patient[];
  upcomingAppointments: Appointment[];
}

type AppAction =
  | { type: 'SET_ONLINE'; payload: boolean }
  | { type: 'SET_SYNCING'; payload: boolean }
  | { type: 'SET_LAST_SYNC'; payload: Date }
  | { type: 'SET_PENDING_COUNT'; payload: number }
  | { type: 'SET_USER'; payload: User }
  | { type: 'SET_PATIENTS'; payload: Patient[] }
  | { type: 'ADD_PATIENT'; payload: Patient }
  | { type: 'UPDATE_PATIENT'; payload: Patient }
  | { type: 'DELETE_PATIENT'; payload: string }
  | { type: 'SET_AT_RISK'; payload: Patient[] }
  | { type: 'SET_APPOINTMENTS'; payload: Appointment[] };

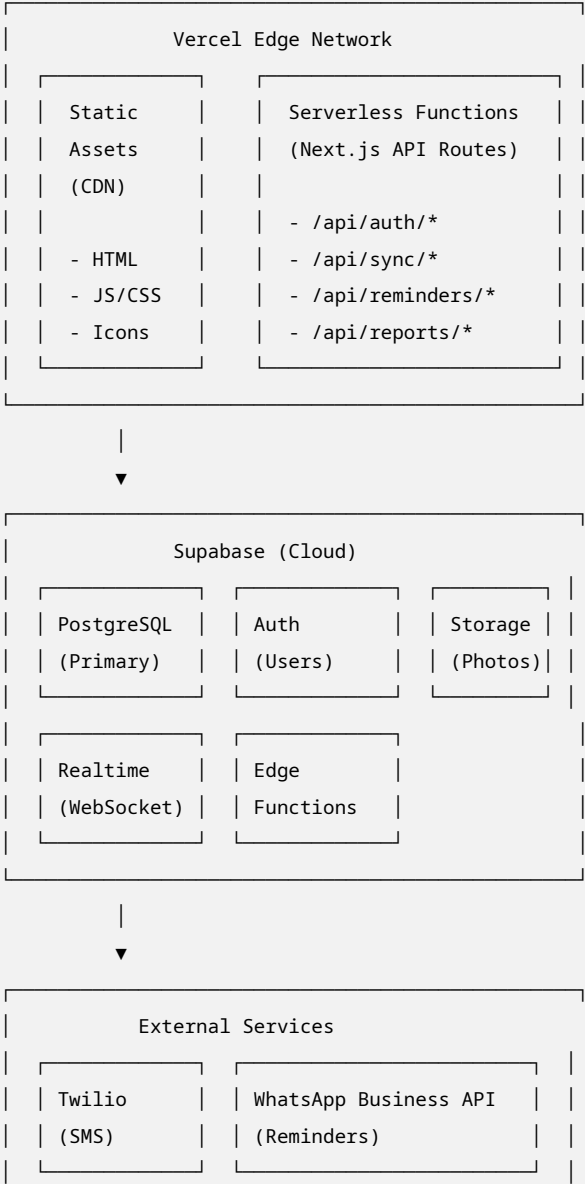
```

8.8 Security Architecture

Security Layers
1. HTTPS (TLS 1.3)
- All network encrypted
2. Authentication
- Supabase Auth (JWT)
- Phone OTP for CHWs

- Session timeout: 15 min
3. Authorization
- Row Level Security (RLS)
- Role-based access (CHW/Nurse)
4. Data Protection
- IndexedDB encryption (SubtleCrypto)
- No PHI in URLs/query params
- Auto-logout on inactivity
5. Audit Trail
- All changes logged with user+time
- Sync logs for accountability

8.9 Deployment Architecture (Vercel)



9. User Flows

9.1 CHW: Register New Patient

Login → Home → "New Patient" → Enter Details → Take Photo (opt) →
Calculate EDD → Save → IndexedDB (instant) → Background Sync (when online)

9.2 CHW: Log Visit + Decision Tree

Patient List → Select Patient → "Log Visit" → Enter Vitals →
"Check Symptoms" → Answer Questions → Get Triage Result →
Add Notes → Save → Auto-flag if at-risk

9.3 CHW: Schedule Appointment

Patient Detail → "Schedule Appointment" → Select Date →
Select Location → Set Purpose → Save → Reminder auto-queued

9.4 Nurse: Review At-Risk Patients

Login (Nurse) → Dashboard → "At-Risk Patients" →
Filter by Severity → Select Patient → View Full History →
Add Nurse Note → Mark as Reviewed

10. UI/UX Design Guidelines

10.1 Design Principles

- **Simplicity:** One primary action per screen
- **Visibility:** Large text (16px+), high contrast
- **Feedback:** Clear success/error states with icons
- **Offline Clarity:** Always show connection status
- **Speed:** Minimize taps, auto-fill where possible

10.2 Color System

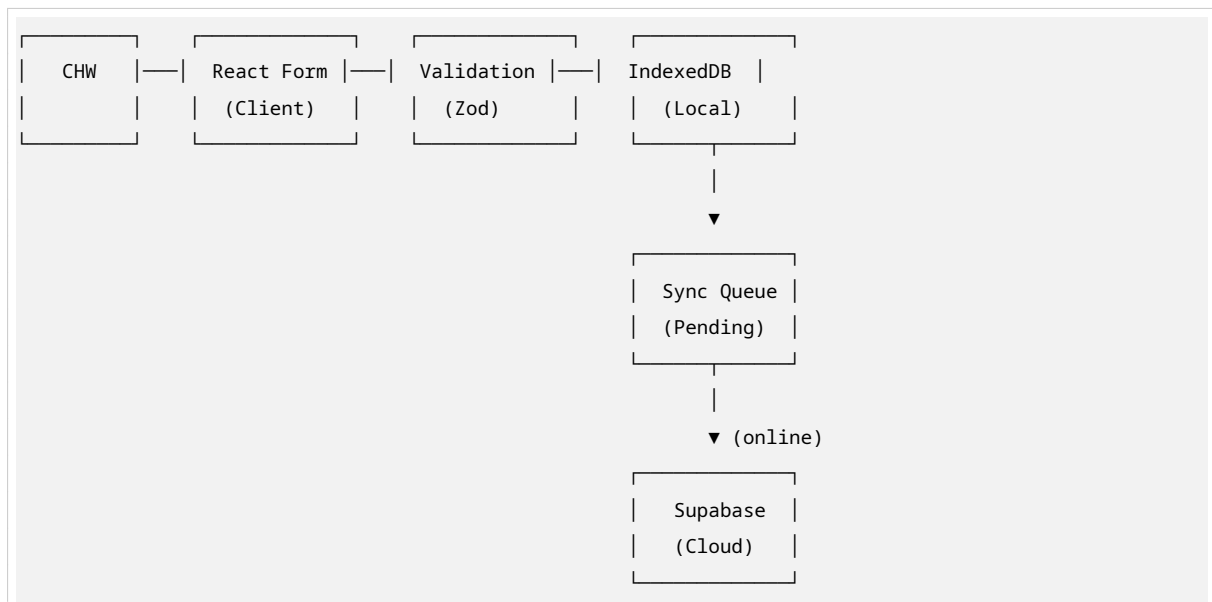
Token	Value	Usage
Primary	#E11D48	Brand, CTAs, active states
Success	#16A34A	Synced, healthy, completed
Warning	#F59E0B	At-risk medium, pending
Danger	#DC2626	At-risk high, emergency, errors
Info	#2563EB	Information, links
Offline	#6B7280	Offline indicator

10.3 Key Screens

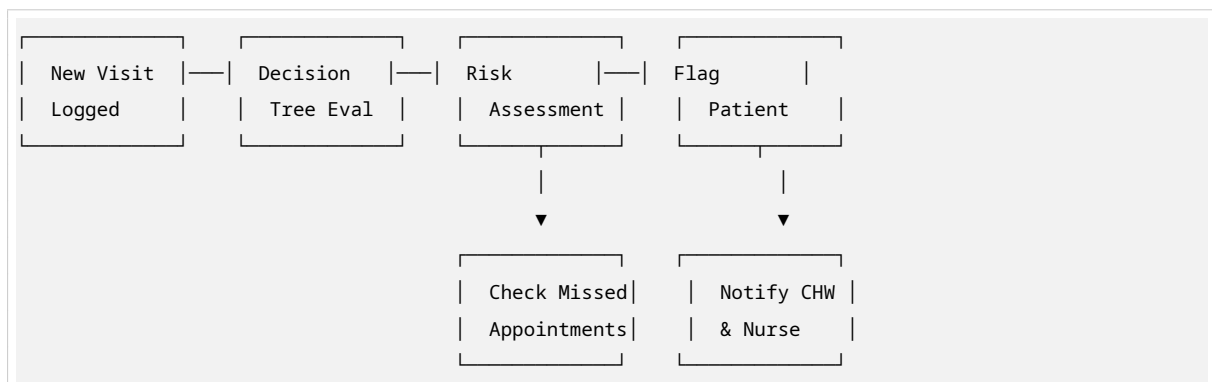
1. **Login:** Phone number input, OTP verification
 2. **CHW Dashboard:** Stats cards, quick actions, at-risk alerts
 3. **Patient List:** Searchable, filterable, risk badges
 4. **Patient Detail:** Profile, history timeline, action buttons
 5. **Visit Form:** Vital signs inputs, symptom checklist
 6. **Decision Tree:** Step-by-step questions, result display
 7. **Appointment Calendar:** Monthly view, color-coded status
 8. **Nurse Dashboard:** Charts, tables, export buttons
-

11. Data Flow Diagrams

11.1 Patient Registration Flow



11.2 At-Risk Detection Flow



12. Testing Strategy

12.1 Test Levels

Level	Focus	Tools
Unit	Components, hooks, utilities	Jest + React Testing Library
Integration	API routes, sync engine	Jest + MSW
E2E	Full user flows	Playwright
Offline	IndexedDB, service worker	Manual + automated
Performance	Load time, sync speed	Lighthouse, WebPageTest

12.2 Critical Test Scenarios

1. Register patient offline → sync when online → verify in Supabase
2. Log visit with red-flag symptoms → verify auto-flagging
3. Miss 2 appointments → verify at-risk escalation
4. Concurrent edit by CHW and nurse → verify conflict resolution
5. App update → verify service worker cache refresh

13. Success Metrics & Analytics

13.1 Product Metrics

Metric	Target	Measurement
Daily Active CHWs	>80% of registered	Login tracking
Patient Registration Rate	>50 new/week per CHW	DB count
Appointment Adherence	>90% attended	Appointment status
At-Risk Detection Time	<24 hours from flag	Timestamp diff
Sync Success Rate	>99%	Sync logs
App Load Time	<3s on 3G	Lighthouse

13.2 Health Outcomes (Long-term)

- Reduction in missed prenatal visits
- Earlier identification of pregnancy complications
- Improved maternal health indicators at district level

14. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Device theft/loss	High	High	Remote wipe, data encryption, no local auth persistence
Prolonged offline period	Medium	Medium	30-day local storage, sync queue persistence
CHW training gap	Medium	High	In-app tutorials, simple UI, supervisor support

Table 9 – continued

Risk	Likelihood	Impact	Mitigation
Network connectivity issues	High	Medium	Offline-first design, retry logic, sync indicators
Data privacy concerns	Medium	High	Encryption, RLS, audit logs, consent forms
Battery drain	Medium	Medium	Efficient sync, background sync only when charging

15. Timeline & Milestones

Week 1: Core CHW Functionality

Day	Task	Deliverable
1-2	Project setup, auth system, IndexedDB schema	Login working, DB initialized
3-4	Patient registration, patient list, patient detail	CRUD for patients
5	Visit logging form, visit history	Visit records working
6	Appointment scheduling, calendar view	Appointments CRUD
7	Decision tree UI, rule engine, risk assessment	Symptom checker working

Week 2: Advanced Features & Dashboard

Day	Task	Deliverable
8	At-risk detection logic, flags, alerts	Risk system working
9	Nurse dashboard UI, patient overview	Dashboard visible
10	Firebase/Supabase sync, conflict resolution	Bidirectional sync
11	SMS/WhatsApp reminder integration	Reminders sending
12	Testing, bug fixes, edge cases	Stable build
13	PWA optimization, performance tuning	Installable PWA

Table 11 – continued

Day	Task	Deliverable
14	Demo preparation, documentation	Demo ready

16. Future Roadmap (Post-MVP)

Phase 2 (Months 2-3)

- Krio language support
- Government approval process initiation
- Basic analytics dashboard for Ministry of Health
- Multi-district support

Phase 3 (Months 4-6)

- Full EMR integration
- Video telemedicine (for district nurses)
- ML-based risk prediction
- Integration with national health ID system
- Community health worker training module

17. Appendix

17.1 Glossary

Term	Definition
CHW	Community Health Worker
EDD	Estimated Delivery Date
PWA	Progressive Web Application
IndexedDB	Browser-based NoSQL database
RLS	Row Level Security
PHI	Protected Health Information
OTP	One-Time Password

17.2 Decision Tree Rules (Sample)

Condition	Triage	Action
Vaginal bleeding at any gestational age	Red	Immediate referral
Severe headache + visual disturbances + BP 140/90	Red	Immediate referral
BP 140/90 without symptoms	Yellow	Refer within 24h
Fever >38°C	Yellow	Malaria test, refer if positive

Table 13 – continued

Condition	Triage	Action
Reduced fetal movement	Yellow	Refer within 24h for assessment
Mild headache, no other symptoms	Green	Routine care, monitor

17.3 Recommended Visit Schedule

Gestational Age	Visit Frequency
0-28 weeks	Every 4 weeks
28-36 weeks	Every 2 weeks
36-40 weeks	Every week
Post-term (>40 weeks)	Every 2-3 days

Document Version: 1.0 Last Updated: June 2026 Author: Product & Engineering Team